Principles of Computer Science II

Reading code and recursion

Marco Zecchini

Sapienza University of Rome

Tutorial 3

Today's plan

1. **Reading code**: a quiz, in three teams. Short pieces of Python on the topics of Tutorials 1 and 2; you predict what they print, or find the bug, *without* running them.
2. **Recursion**: the exercises we did not get to in Tutorial 2, pseudocode first, then Python.

Why a quiz? So far you have *written* code. In this course you will mostly *read* it: the algorithms of the book, the code of your colleagues, your own code when it does not work. Reading is a skill of its own, and it needs practice.

Rules of the game

- We split into **3 teams**. Each team needs one sheet of paper and *no* computer for Part 1.
- For every question: **around 3/4 minutes**. The team agrees on **one answer** among (A)–(D) and on a **one-sentence reason**: "it prints X because . . . ".
- When the time is up, every team shows its answer; one team explains, the other two agree or object.
- Then we run the code together and see who was right. One point per correct answer, one more for a correct reason.
- The questions get harder: the last ones are about recursion, the topic of Part 2.

How to read a piece of code

1. **Start from the top** and execute it in your head, one line at a time, in the order Python would.
2. **Keep a table** of the variables on paper: one column per variable, one row per step. Update it at every assignment.
3. **Loops**: write down the value of the loop variable at the first iteration and at the last one; count the iterations.
4. **Functions**: a call jumps into the body with the arguments; `return` jumps back with the result; `print` only shows something.
5. **Types**: is this value a number, a string, a list? Does `/` give a decimal number? Does `input` give a string?

When you find a bug, say *which line* is wrong and *what* it should be.

Question 1: two variables

```
1 a = 5
2 b = a
3 a = a + 1
4 print(a, b)
```

What does it print?

A. 6 6
B. 6 5
C. 5 5
D. An error at line 3: a is used before it is defined.

Question 1: answer

Answer: (B) 6 5

after line	*a*	*b*
1	5	–
2	5	5
3	6	5

- Line 2 stores the *value* of `a` in `b`: from then on the two variables live their own life.
- Line 3 is not an equation: Python computes the right-hand side `(5 + 1)` *first*, then stores it in `a`.

Question 2: spot the bug

The function should return $1 + 2 + \cdots + n$, like SumIntegers of Tutorial 1.

```
1 def sum_integers(n):
2     total = 0
3     for i in range(1, n):
4         total = total + i
5     return total
6
7 print(sum_integers(5))
```

What does it print, and which line is wrong?

A. 15, nothing is wrong
B. 10
C. 14
D. 0

Question 2: answer

Answer: (B) 10, the bug is at line 3

`range(1, n)` produces $1, 2, 3, 4$: the last value is **excluded**, so $n = 5$ is never added and the result is 10 instead of 15.

- "for $i \leftarrow 1$ to n" in pseudocode is `range(1, n + 1)` in Python (cheat sheet of Tutorial 1).
- This is an *off-by-one* error: the program runs, gives a plausible number, and it is wrong. The only way to catch it is to test on an input whose answer you know.
- Check loops at the **boundaries**: what is `i` at the first iteration? At the last?

Question 3: a while loop

```
1 n = 1
2 t = 0
3 while n < 10:
4     n = n * 2
5     t = t + 20
6 print(n, t)
```

What does it print?

A. 8 60
B. 10 80
C. 16 80
D. It never stops.

Question 3: answer

Answer: (C) 16 80

iteration	`n < 10?`	n	t
start		1	0
1	yes	2	20
2	yes	4	40
3	yes	8	60
4	yes	16	80
5	no: stop		

- The condition is checked *before* each iteration: with $n = 8$ the body runs once more, and n ends at 16, not at 10.
- This is Exercise 4 of Tutorial 1 (bacteria doubling every 20 minutes): the first time the population reaches 10 cells is after 80 minutes.

Question 4: spot the bug

The function should print `3 2 1` on separate lines, then `Go!`.

```
1 def count_down(n):
2     while n > 0:
3         print(n)
4     print("Go!")
5
6 count_down(3)
```

What does it actually do?

A. Prints `3 2 1 Go!` as expected.

B. Prints 3 forever.

C. Prints only `Go!`.

D. An error: `n` is not defined.

Question 4: answer

Answer: (B) 3, forever

The body of the `while` never changes `n`, so the condition `n > 0` stays true: an **infinite loop**. The fix is a line `n = n - 1` inside the loop, after the `print`.

- "The body must eventually make the condition false" (Tutorial 1). Before writing a `while`, ask: which variable moves towards the exit?
- In Colab an infinite loop looks like a cell that never finishes: stop it with the square button, or `Runtime > Interrupt execution`.
- Remember this one for Part 2: a recursive call that does not shrink the problem is the same bug in disguise.

Question 5: print or return?

```
1 def double(x):
2     print(x * 2)
3
4 y = double(3)
5 print(y)
```

What does it print?

A. 6 then 6
B. 6 then `None`
C. 6 only
D. An error at line 4: the function returns nothing.

Question 5: answer

Answer: (B) 6 then `None`

Line 2 *shows* 6 on the screen, but the function has no `return`: its result is the special value `None`, which is what ends up in `y`.

- `print` is for humans; `return` is for the rest of the program. A function whose result you need must `return` it.
- A typical symptom of this bug: `double(3) + 1` fails with `TypeError: unsupported operand ... 'NoneType' and 'int'`.
- The tests of the Colab notebooks check the *returned* value: a function that only prints fails them.

Question 6: decrement a list

```
1 a = [5, 3, 8]
2 for x in a:
3     x = x - 1
4 print(a)
```

What does it print?

A. `[4, 2, 7]`
B. `[5, 3, 8]`
C. `[5, 3, 8, 4, 2, 7]`
D. An error at line 3: you cannot assign to `x` inside a loop.

Question 6: answer

Answer: (B) `[5, 3, 8]`

At each iteration `x` receives a *copy* of an element; line 3 changes the copy, and the list is untouched (Exercise 3 of Tutorial 2).

```
for i in range(len(a)):
    a[i] = a[i] - 1        # a is now [4, 2, 7]
```

- To change the elements of a list you need their **index**: `a[i] = ...`.
- Or build a new list with `append` and return it.

Question 7: two names for a list

```
1 a = [1, 2, 3]
2 b = a
3 b.append(4)
4 print(a, len(b))
```

What does it print? (Compare with Question 1.)

A. `[1, 2, 3] 4`
B. `[1, 2, 3] 3`
C. `[1, 2, 3, 4] 4`
D. An error at line 3: `append` returns nothing.

Question 7: answer

Answer: (C) `[1, 2, 3, 4] 4`

Line 2 does *not* copy the list: `a` and `b` are two names for the **same** list, so what you append through `b` is visible through `a`.

- With numbers (Question 1) `b = a` copies the value, because a number cannot be modified: `a + 1` is a *new* number.
- A list can be modified in place (`append`, `a[i] = ...`): every name attached to it sees the change. This is why "modify in place" in Tutorial 2 means "whoever called you sees the change".
- A real copy: `b = a.copy()` or `b = a[:]`.

Question 8: spot the bug

The function should return `{'A': 1, 'C': 1, 'G': 1, 'T': 1}` (bonus of Exercise 4, Tutorial 2).

```
1 def count_bases(dna):
2     counts = {}
3     for base in dna:
4         counts[base] = counts[base] + 1
5     return counts
6
7 print(count_bases("ACGT"))
```

What happens?

A. It prints `{'A': 1, 'C': 1, 'G': 1, 'T': 1}`.
B. It prints `{}`.
C. `KeyError: 'A'` at line 4.
D. `TypeError` at line 4: you cannot add 1 to a dictionary.

Question 8: answer

Answer: (C) `KeyError: 'A'`

At the first iteration the dictionary is empty: reading `counts['A']` on the right-hand side of line 4 looks up a key that does not exist yet.

```
if base in counts:
    counts[base] = counts[base] + 1
else:
    counts[base] = 1
```

- Python evaluates the right-hand side *before* the assignment, exactly as in Question 1.
- Alternative: start from `counts = {'A': 0, 'C': 0, 'G': 0, 'T': 0}` (but then `"N"` raises again).

Question 9: reading a file

```
1 with open("seqs.txt", "w") as f:
2     f.write("ATGCGC\n")
3     f.write("GGA\n")
4 with open("seqs.txt", "r") as f:
5     for line in f:
6         print(len(line), end=" ")
```

What does it print?

A. 6 3
B. 7 4
C. 9
D. 11

Question 9: answer

Answer: (B) `7 4`

Each line read from the file still contains the newline character `"\n"` at the end: `"ATGCGC\n"` has 7 characters. `end=" "` keeps the two numbers on the same line.

- This is why Tutorial 2 always did `dna = line.strip()` before using a line: without it, the "last nucleotide" of every sequence is a newline, and `len`, `count_nucleotides`, comparisons are all off by one.
- A mode `"w"` empties the file before writing: run the cell twice and the file still has two lines.

Question 10: in which order?

```
1 def show(n):
2     if n == 0:
3         print("go")
4         return
5     show(n - 1)
6     print(n)
7
8 show(3)
```

What does it print (one value per line)?

A. 3 2 1 go
B. go 3 2 1
C. go 1 2 3
D. 1 2 3 go

Question 10: answer

Answer: (C) `go 1 2 3`

`show(3)` calls `show(2)` *before* printing anything; `show(2)` calls `show(1)`, which calls `show(0)`: that one prints `go` and returns. Now the pending calls finish, innermost first: `show(1)` prints `1`, then `show(2)` prints `2`, then `show(3)` prints `3`.

- Every call *waits* for the inner one at line 5 and continues at line 6 only when it is back.
- Swap lines 5 and 6 and the output becomes `3 2 1 go`: what happens *before* the recursive call goes down, what happens *after* it comes back up.

Question 11: a recursive count

```
1 def count_a(s):
2     if s == "":
3         return 0
4     if s[0] == "A":
5         return 1 + count_a(s[1:])
6     return count_a(s[1:])
7
8 print(count_a("ACAGTA"))
```

What does it print?

A. 6
B. 3
C. 0
D. `RecursionError`: the string never becomes empty.

Question 11: answer

Answer: (B) 3

Every call looks at the *first* character and asks the rest of the string, `s[1:]`, how many `A`s it contains: if the first character is an `A` it adds 1 to that answer, otherwise it passes it on unchanged.

- Chain of calls: `"ACAGTA"` → `"CAGTA"` → `"AGTA"` → `"GTA"` → `"TA"` → `"A"` → `""`; the string loses one character per call, so it *does* reach the base case.
- Coming back up: $0 \to 1 \to 1 \to 1 \to 2 \to 2 \to 3$.
- "Smaller" does not have to mean $n - 1$: here it is a shorter string. Keep this in mind for Exercise 4.

Question 12: spot the bug

```
1 def sum_to(n):
2     if n == 1:
3         return 1
4     return n + sum_to(n - 1)
5
6 print(sum_to(4))
7 print(sum_to(0))
```

What does it print?

A. 10 then 0
B. 10 then 1
C. 10 then `RecursionError`
D. `RecursionError` right away: the function calls itself.

Question 12: answer

Answer: (C) `10` then `RecursionError`

`sum_to(4)` $= 4 + 3 + 2 +$ `sum_to(1)` $= 10$. But `sum_to(0)` calls `sum_to(-1)`, then `sum_to(-2)`, ...: the base case $n = 1$ is **never reached** and Python gives up after about 1000 nested calls.

- The bug is the base case at line 2: it should be `n == 0`, returning `0`. Then `sum_to(0)` is `0` and `sum_to(4)` is still `10`.
- Two questions to ask of every recursive function: *is there a base case?* and *does every call move towards it?*
- Same bug as the infinite loop of Question 4: nothing moves towards the exit.

Lessons from the quiz

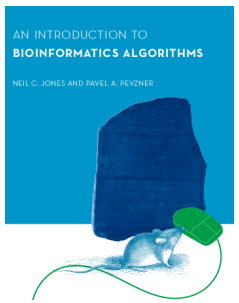
Trap	Question
the right-hand side is computed first, then assigned	1, 8
`range(a, b)` stops at $b - 1$	2
a `while` checks the condition before each iteration	3
the loop body must move towards the exit	4, 12
`print` shows, `return` gives back	5
`for x in a` gives copies; `b = a` does not copy a list	6, 7
a line read from a file ends with `"\n"`	9
before the recursive call: going down; after it: coming up	10
smaller input: a shorter string, `s[1:]`	11

Recursion: reminder from Lecture 2

"Recursion is one of the most ubiquitous algorithmic concepts. Simply, an algorithm is recursive if it calls itself." (p. 24)

- It has to terminate: a **base case**, an input whose answer is known without further calls.
- Every call works on a **smaller** instance, moving towards the base case: the algorithm "bottoms out" (p. 31).
- "Recursive algorithms can often be rewritten to use iterative loops instead, and vice versa; it is a matter of elegance and clarity" (p. 28).

Jones, Pevzner, Sections 2.5–2.6, pp. 24–33.



Recursion: pseudocode and Python

Factorial(n)

```
1 if n = 1
2     return 1
3 return n · Factorial(n - 1)
```

```
def factorial(n):
    if n == 1:
        return 1
    return n * factorial(n - 1)
```

- The pseudocode of the book may call itself, like a Python function: the translation is the same as in Tutorial 1.
- Line 1 is the **base case**; line 3 calls the algorithm on a **smaller** input, $n - 1$ (Question 12: what if it is never reached?).

How to work on the exercises

For each exercise, in this order:

1. identify the **base case** and write it down;
2. write the **recursive case**: the answer for the input in terms of the answer for a smaller input;
3. write the **pseudocode**, in the style of the book;
4. write the **Python** function and test it on the examples;
5. write the **chain of calls** for one small input, as in Question 10, and check that it reaches the base case;
6. (bonus) write the **iterative** version with a loop and compare the two.

Keep working in your team: one writes, the others read, as in Part 1.

Part 2: open the notebook

Colab notebook of Part 2

`https://drive.google.com/file/d/1O70Iem2Acp-tEMnBy3LZf6DKj81d3yc_/view?usp=sharing`

1. Open the link (also on the course website); you need a Google account.
2. `File > Save a copy in Drive`, otherwise your changes are lost.
3. Every exercise has a cell to complete and a cell of **tests**: run the tests to check your solution.
4. The twelve snippets of Part 1 are in a separate notebook: run them at home.

Exercise 1: power

- Write a recursive function `power(x, n)` that returns x^n for an integer $n \geq 0$, without using `**`.
- Examples: `power(2, 10)` returns `1024`; `power(5, 0)` returns `1`.
- Hint: $x^n = x \cdot x^{n-1}$. What is x^0?
- Chain of calls for `power(2, 3)`:
  $(2, 3) \to (2, 2) \to (2, 1) \to (2, 0)$. Which argument shrinks?

Exercise 2: sum of the digits

- Write a recursive function `sum_digits(n)` that returns the sum of the digits of a non-negative integer n.
- Examples: `sum_digits(1234)` returns 10; `sum_digits(7)` returns 7.
- Hint: `n % 10` is the last digit and `n // 10` is the number without it: $1234 = 123 \cdot 10 + 4$. The base case is a number with one digit.
- Question: the smaller input here is not $n - 1$. Does every call still move towards the base case? Why?

Exercise 3: greatest common divisor

Euclid's observation: the greatest common divisor of a and b, with $b > 0$, is the same as that of b and a mod b (the remainder, `a % b`); and $\gcd(a, 0) = a$.

- Write a recursive function `gcd(a, b)`: `gcd(48, 18)` returns 6; the calls are $(48, 18) \to (18, 12) \to (12, 6) \to (6, 0)$.
- Write a program that reads two integers from the keyboard with `ask_integer` of Tutorial 2 (Exercise 6) and prints their gcd.
- Question: does every call make the problem smaller? Which argument shrinks? What happens with `gcd(18, 48)`?

Exercise 4: transcribing DNA into RNA (Rosalind)

Problem RNA on Rosalind: given a DNA string t, return its transcription, obtained by replacing every `T` with `U`.

- Example: `GATGGAACTTGACTACGTAAATT` becomes `GAUGGAACUUGACUACGUAAAUU`.
- Write a recursive `transcribe(t)`: the base case is the empty string; otherwise transcribe the first character and append the transcription of the rest, `t[1:]`.
- Then download the dataset from Rosalind, read it from the file (Tutorial 2, remember `strip()`), write the answer to a file and submit it.
- Question: what happens with a string of 10 000 nucleotides? Try it, read the error, and write the iterative version (a loop, or `t.replace("T", "U")`).

Recursion: cheat sheet

Step	**Question to ask**
base case	which input has an answer I already know?
recursive case	how does the answer follow from that of a smaller input?
termination	does *every* call move towards the base case?
order	before the call: going down; after the call: coming up
limits	deeper than about 1000 calls? `RecursionError`: use a loop

Smaller does not always mean $n - 1$: a shorter string (`t[1:]`), fewer digits (`n // 10`), a smaller remainder (`a % b`).

What's next

- At home: `sum_to(n)` of Question 12, fixed, and the iterative version of all four exercises. Compare: same answers, which one is easier to read?
- Rosalind: submit *Transcribing DNA into RNA*; then *Complementing a Strand of DNA*, with `complement` of Tutorial 1 and a loop over the string backwards.
- Recursion will come back with sorting (merge sort, quick sort), dynamic programming and graphs: the recipe stays the same.